COMP20005 Workshop Week 3

	Any problem from lectures? How about: • expressions (check with **`Ex 3.01`**) • **`if`** statement (check with **`Ex 3.02`**) • safe use of **`scanf`** with **`if`** Do-Together **`3.06`**
LAB	Finish **`Ex 3.06`** if not done Design and implement a solution to **`Ex 3.03, 3.07, 3.07b`**. Make sure to get feedback from Anh for **`3.07`** or **`2.08`** at least Other exercises from **`Chapter 3`**

the operator precedence in expressions

order of precedence

++, -- !, -, (*type*)	unary
*, /, % +, -	arithmetic
<, >, <=, >= ==, !=	relational
&& ||	logical
=, +=, *=, etc	assignments (right to left)

- Do Ex 3.01

if . . .

```
if ( guard ) {
  // do some stuffs A1
}
// do B
```

```
if ( guard ) {
  // do some stuffs A1
} else {
  // do alternatives A2
}
// do B
```

Suppose that a, b, c were declared as int and had some value. Write a code segment to compute min2, max3, where:

- min2 is the min of a, b
- max3 is the max of a, b, c.

if (*guard*)... : a *guard* can be any expression

	our interpretation	C-Machine interpretation
guard	a *condition* that is either TRUE or FALSE	any *expression;* non-zero is TRUE, zero is FALSE
values like: 6, 0.0, a	Numeric, can be: • non-zero • zero	A value is an expression. As a ***guard***: • non-zero value means **TRUE** • zero means **FALSE**.
comparisons: <, <=, >, >=, ==, !=	a comparison like **a<b** yields: • TRUE (or **1**) if correct • FALSE (or **0**) if incorrect	An expression of the form of: • a comparison (e.g., a<b) • a logic expression (e.g., (a>3) && !(a>10)) yields an *integer* value: • 1 if the expression is TRUE • 0 if the expression is FALSE
logical operations: !, &&, ||	yields value TRUE/**1** or FALSE/**0**	
Special Note: an assignment is also an expression.		
assignment a= b + 5	a gets the current value of b+5	• a is assigned the current values of b+5 • a=b+5 is an *expression* with the value b+5

Ex 3.2 a): (use Ed and/or pens & papers)

Trace the action of these statements, and determine the values printed out by each of the printf *statements. Assume that all variables have been declared to be of type* int.

```
i = 3;  j = 4;
if (  i<j  &&  j<6  ) {
    i= i+j;
} else {
    j= i+j;
}
printf ("i = %d, j = %d\n", i, j);
```

i =, j=

3.2 b)

```
int i = 3, j = 4, k = 7;
if ((i < j || j < k) && j < i) {
    i = i + 1;
    if (i * i > k) {
        k = k + 1;
    }
} else {
    j = j + 1;
    if (i * i > k) {
        k = k + 2;
    }
}
printf("i = %d, j = %d, k = %d\n", i, j, k);
```

i = , j = , k =

3.2 c)

```
1  int month = 7, days;
2  if (month == 2) {
3      days = 28;
4  } else if (month == 4 || 6 || 9 || 11) {
5      days = 30;
6  } else {
7      days = 31;
8  }
9  printf("days = %d\n", days);
```

```
days =
```

3.2 d)

```
int x = 1, y = 2;
if (x > y)
    printf("x = %d, y = %d\n", x, y);
    x = x + 1;
if (x < y)
    printf("x = %d, y = %d\n", x, y);
    y = y + 2;
printf("x = %d, y = %d", x, y);
```

```
x = , y =
```

3.2 e)

```
1   int x = 1, y = 2;
2   if (x > y); {
3       printf("x = %d, y = %d\n", x, y);
4       x = x + 1;
5   }
6   if (x < y); {
7       printf("x = %d, y = %d\n", x, y);
8       y = y + 2;
9   }
10  printf("x = %d, y = %d\n", x, y);
```

```
x = , y =
???
```

3.2 f)

```
x = 0; y = 0;
if (y<x) {
    printf (“y is smaller\n”);
} else if (y=x) {
    printf (“x and y are equal\n”);
} else {
    printf (“y is greater\n”);
}
```

???

How scanf works?

scanf is a **formatted input** function: it reads characters from the input stream (usually stdin) and **matches them against a format string**.

Input Buffer

When you type *and press **Enter***, the characters are placed in an **input buffer**. scanf processes this buffer **one character at a time**.

Whitespace Handling

Most specifiers (%d, %f, %s) **skip leading whitespace** (spaces, tabs, newlines). Exceptions: %c and %[...] do **not skip whitespace** and read the **next character exactly as it appears**.

Matching

After skipping whitespace (if applicable), scanf attempts to match the input to the specifier.

Success: %d with input 42 consumes both digits.

Failure: %d with input abc stops immediately when a is encountered.

How scanf works?

```
int converts, n= 0;    double f= 0.0;
converts= scanf("%d %lf", &n, &f);
```

The process of executing the above scanf:

- **Skip:** Automatically ignores leading whitespace (spaces, tabs, newlines) for %d, %f,%lf, and %s.
- **Match:** Consumes characters that fit the specifier. It **stops** at the first non-matching character.
- **Return:** Outputs the total number of variables successfully assigned.

Examples

Input Buffer	n	f	return value (converts)	Buffer After
10 2.5 777 ⏎	10	2.5	2	777 ⏎
ten 2.5⏎	0	0.0	0	ten 2.5⏎
10 two⏎	10	0.0	1	two⏎
102.5⏎	102	0.5	2	⏎

Validation Pattern: Always check the return value to ensure the user provided the expected data:

```
if ( scanf("%d %lf", &n, &f) != 2 ) {
  printf("Invalid input\n");
  exit(EXIT_FAILURE);
}
```

Quiz 1

What is the output of the following fragment:

```
int a=1, b=2;
if ( a = b ) {
  printf("a= %d ", a);
} else {
  printf("b= %d", b);
}
printf("\n");
```

A: a= 1 b= 2	**B:** a= 1
C: a= 2	**D:** b= 2

Quiz 2

What **XXX** *should be in the following fragment:*

```
printf("Enter value for a, b, c: ");
if ( scanf("%d%d%d", &a, &b, &c) XXX ) {
  printf("Please enter 3 integers\n");
  exit( EXIT_FAILURE );
}
```

A: `!= 2`	**B:** `!= EOF`
C: `!= 3`	**D:** `== 3`

Quiz: Check your answers

Quiz 1: C. a= 2

Why? The expression a = b inside the if statement is an **assignment**, not a **comparison** (a == b).

In this case, a = b assigns the value of b (which is 2) to a. Since the result of the assignment is 2 (a non-zero value), it is evaluated as **TRUE**.

Quiz 2: C. != 3

Why? The scanf call seeks three values; therefore, it should return 3 if successful. Any other result implies that the execution failed.

.

Compare:

	a= 5;
	b= 5;
		a= b= 5; // same as a= (b= 5);
		Assignment is an expression!

	a= a * b;
	n= n+1;
	n += 1;
	m= m-1;
		a *= b;
		n++;
		m--;

	scanf("%d%d", &a, &b);
	
	//rest of the program
		if (scantf("%d%d", &a, &b) != 2) {
		  printf("invalid input\n");
		  exit(EXIT_FAILURE);
		}
		// rest of the program

LAB

Minimum:

- Do-together **`Ex 3.06`**
- Ask Anh to comment on your **`Ex 2.08`** if he failed to do so for you last week
- Finish **`Ex 3.06`** if not done
- Complete **`Ex 3.07:`** Temperature converter

Additional:

- Complete **`Ex 3.07b`**: Generalised converter (first, copy 3.07 to 3.07b, then update it).
- Complete **`Ex 3.03:`** Quadratic roots
- Complete other exercises from **`Chapter 3`**

Wrap Up

- in expressions, operator precedence is as expected: *unary*, *arithmetic*, *relational*, *logic*, *assignment*
- *Relational* and *Logic* Operations yield integer *1* (for *TRUE*) and *0* (for *FALSE*)

```
if ( guard ) { ... } else { ... }
```

guard : any expression,

- non-zero is TRUE,
- zero is FALSE

safe reading some variables

```
if (scanf(... ) != <number of variables>) {
   printf("...\n");
   exit(EXIT_FAILURE);
}
// process the input tuple
```